Chapter 3: Protein Representations & Structure Engineering

Chapter 3: Protein Representations & Structure Engineering

Chapter 1 introduced the two modalities this book covers; Chapter 2 built the computational toolkit for one of them, small molecules. This chapter builds the analogous toolkit for the other: proteins. Everything here — sequence representations, 3D structural anatomy, and the machine-learning-ready representations built from both — is infrastructure the rest of Part III (protein language models, structure prediction, de novo design) and the docking chapters in Part IV depend on directly.

3.1 Sequences & Amino Acid Properties

FASTA format

A protein sequence is, at its simplest, a string over a 20-letter alphabet (the standard amino acids, plus a handful of extended-alphabet codes for ambiguity and non-standard residues). The near-universal plain-text format for storing one or more such sequences is **FASTA**, named after the FASTA sequence-alignment tool it originated with (Pearson & Lipman, 1988). A FASTA record is a single header line beginning with `>`, followed by the sequence itself, wrapped at an arbitrary line width:

```
>sp|P00533|EGFR_HUMAN Epidermal growth factor receptor
MRPSGTAGAALLALLAALCPASRALEEKKVCQGTSNKLTQLGTFEDHFLSLQRMFNNCE
VVLGNLEITYVQRNYDLSFLKTIQEVAGYVLIALNTVERIPLENLQIIRGNMYYENSYA
...
```

Every major sequence database (UniProt, NCBI, Ensembl) and every tool this book uses that consumes a raw sequence (BioPython, ESM, AlphaFold) either accepts or emits FASTA. It has no formal specification body — it is a de facto standard defined by decades of tool compatibility — which is precisely why its simplicity matters: a format with essentially zero parsing ambiguity is what let it become universal infrastructure.

Amino acid properties

The 20 standard amino acids share a backbone (an amino group, a carboxyl group, and a central α-carbon) and differ only in their side chain (R group), and it is the physicochemical character of that side chain — charge, polarity, hydrophobicity, size, and the presence of distinguishing groups like proline's ring-closed backbone or cysteine's thiol — that determines nearly everything downstream: which secondary structures a residue favors (§3.2), which residues cluster in a folded protein's hydrophobic core versus its solvent-exposed surface, and which positions in a binding pocket (§3.4) a small molecule can hydrogen-bond to versus merely pack against. Substitution matrices (below) are, in effect, a quantitative encoding of exactly this: how interchangeable two amino acids are without disrupting a protein's structure or function.

Substitution matrices

Aligning two sequences requires a way to score whether substituting one amino acid for another at a given position is "cheap" (conservative, likely to preserve function) or "expensive" (radical, likely disruptive) — identity alone is too crude, since a leucine-to-isoleucine substitution is usually far more tolerable than a leucine-to-aspartate one. A **substitution matrix** gives every one of the 20×20 amino acid pairs a log-odds score:

$$s(a, b) = \frac{1}{\lambda} \ln \frac{p_{ab}}{q_a\, q_b}$$

where p_{ab} is the observed frequency with which residues a and b are aligned with each other in a trusted reference set of alignments, q_a and q_b are their independent background frequencies, and λ is a scaling constant. Intuitively: if a and b co-occur in real alignments more often than their individual frequencies would predict by chance, $s(a, b) > 0$ (a favorable substitution); if less often, $s(a, b) < 0$. The two matrix families in near-universal use differ in how the reference alignments were built: the older PAM matrices extrapolate substitution rates from very closely related sequences using an explicit evolutionary model, while the BLOSUM matrices (Henikoff & Henikoff, 1992) are derived directly from observed substitution frequencies in blocks of already-aligned, more divergent sequence families — BLOSUM62 (built from blocks with ≤62% pairwise identity) is the default in most modern alignment tools, including the pairwise aligner used below.

Pairwise and multiple sequence alignment

Given a substitution matrix and a gap penalty, computing the highest-scoring alignment between two sequences is a dynamic programming problem. Needleman & Wunsch (1970) gave the algorithm for **global** alignment (align the sequences end-to-end); Smith & Waterman (1981) gave the variant for **local** alignment (find the highest-scoring aligned subsequence, ignoring unrelated flanking regions) by allowing the recurrence to reset to zero rather than go negative. Both remain exactly correct and exactly this: dynamic programming over an $(n + 1) \times (m + 1)$ score matrix, in $O(nm)$ time. A minimal, exact global alignment using BLOSUM62 (BioPython, no external tools required):

```python
from Bio import Align
from Bio.Align import substitution_matrices

aligner = Align.PairwiseAligner()
aligner.substitution_matrix = substitution_matrices.load("BLOSUM62")
aligner.open_gap_score = -10
aligner.extend_gap_score = -0.5

alignment = aligner.align("MKTAYIAKQR", "MKTAYIAKER")[0]
print(alignment)
print("score:", alignment.score)
```

Multiple sequence alignment (MSA) — aligning three or more sequences simultaneously — is what makes cross-species conservation analysis, phylogenetics, and (critically, for Chapter 8) protein language model training data possible, but it is not simply pairwise alignment run $\binom{n}{2}$

times: exact multi-sequence dynamic programming is exponential in the number of sequences, so practical tools use a **progressive alignment** heuristic — build a guide tree from pairwise similarities, then align sequences and sub-alignments along that tree from the leaves inward. Clustal Omega (Sievers et al., 2011) is the present-day standard implementation of this approach, capable of scaling to datasets of hundreds of thousands of sequences. This book does not implement an MSA tool from scratch — that would mean re-deriving production bioinformatics software with no pedagogical benefit over using Clustal Omega or MAFFT directly — but §3.4 does implement the pairwise dynamic-programming primitive that progressive alignment is built from, and Chapter 8 revisits MSA directly as an input to protein language models.

3.2 3D Structural Anatomy

File formats: PDB and PDBx/mmCIF

A protein's 3D structure — the (x, y, z) coordinate of every resolved atom, plus experimental metadata (resolution, method, the identity of any bound ligands) — is distributed by the Protein Data Bank in two formats: the original fixed-column-width `.pdb` format, and its successor PDBx/mmCIF, the wwPDB's extensible standard format since 2014 (wwPDB consortium, 2019; see also Chapter 1 §1.4). This book's hands-on projects use the legacy `.pdb` format where possible for readability — it is plain text, and a `HETATM` line for a bound ligand or an `ATOM` line for a protein atom is human-inspectable without a parser — but `.pdb`'s fixed-width columns cannot represent structures with more than 62 chains or 99,999 atoms, which is why mmCIF is now the archive's primary format for large assemblies. Both are parsed identically by BioPython's `Bio.PDB` module, which is what this chapter's hands-on project (§3.4) uses.

Backbone dihedral angles: phi and psi

A polypeptide backbone's local geometry is almost entirely described by two torsion (dihedral) angles per residue: **phi** (ϕ), the rotation around the N–Cα bond, and **psi** (ψ), the rotation around the Cα–C bond. (A third backbone dihedral, omega, rotates around the peptide C–N bond itself, but that bond's partial double-bond character from amide resonance locks it planar — almost always at 180°, trans — so it is rarely treated as a free variable.) Not all (ϕ, ψ) combinations are physically possible: many would force atoms into steric clashes. Ramachandran, Ramakrishnan, and Sasisekharan (1963) computed which combinations are sterically allowed from hard-sphere atomic radii alone, producing the **Ramachandran plot** — a (ϕ, ψ) scatter plot whose allowed regions correspond almost exactly to the dihedral angles observed in real, experimentally solved structures decades later. Two regions dominate: right-handed **α-helix** (Pauling, Corey, & Branson 1951 first proposed this hydrogen-bonded helical backbone conformation on stereochemical grounds, before any protein structure had been solved by crystallography), centered near $\phi \approx -57°, \psi \approx -47°$, and extended **β-strand**, centered near $\phi \approx -120°, \psi \approx +120°$. §3.4′s hands-on project computes real (ϕ, ψ) values from a solved structure and uses simplified versions of these two regions as a coarse, geometry-only secondary-structure classifier.

Secondary and tertiary structure

Secondary structure — the local, repeating backbone conformations (α-helix, β-sheet/strand, and everything else lumped as "coil" or "loop") — is what phi/psi angles proxy for geometrically, but the field's actual standard for assigning it is DSSP (Kabsch & Sander, 1983), which uses backbone hydrogen-bonding patterns (an α-helix's characteristic $i \to i + 4$ backbone N–H⋯O=C hydrogen bond; a β-sheet's inter-strand hydrogen bonding) rather than dihedral geometry alone. This matters pedagogically: the phi/psi-only classifier built in §3.4 is a legitimate, useful approximation — computable with zero extra dependencies directly from atomic coordinates — but it is not DSSP, and should not be reported as equivalent to it in any context where the distinction matters (this caveat is stated in the code itself, not just here). **Tertiary structure** is the full 3D fold: how secondary structure elements pack together in space, stabilized by the same hydrophobic-core-versus-solvent-exposed-surface logic introduced in §3.1′s discussion of amino acid properties. **Quaternary structure**, one level up, is how multiple folded chains (subunits) assemble into a functional complex — not covered further here, but relevant wherever this book discusses antibody structures (Part III) or protein-protein docking.

3.3 Machine Learning Representations

Raw atomic coordinates are not, by themselves, a convenient input to most machine learning models: they are unordered across chains, arbitrary in their global rotation and translation (the same protein measured or modeled twice will not have identical coordinates unless one is deliberately superposed onto the other), and far higher-dimensional than the information content of the fold actually requires. Three representations, in increasing order of the amount of 3D detail they discard, recur throughout the rest of this book:

- **Residue contact maps.** An $N \times N$ binary (or continuous, distance-valued) matrix recording whether each pair of residues is within some cutoff distance of each other — computed in §3.4 as Cα–Cα distance thresholded at 8 Å, a standard convention, though finer-grained variants use the closest side-chain heavy atom instead. A contact map is invariant to global rotation/translation by construction (it depends only on pairwise distances) and was, for decades, the target representation for structure-prediction models before end-to-end 3D generation became tractable: Senior et al. (2020) — AlphaFold's first iteration — worked by predicting a distribution over inter-residue distances directly from sequence and co-evolution features, then using that predicted contact/distance map as a constraint for a downstream folding optimization. AlphaFold2 (Jumper et al., 2021; Chapter 1 §1.2, Chapter 9) replaced this two-stage contact-map-then-fold pipeline with direct end-to-end 3D coordinate prediction, but contact and distance maps remain a standard evaluation representation and a lightweight input feature throughout the field.
- **Molecular surface meshes.** A triangulated approximation of a protein's solvent-accessible or solvent-excluded surface — the boundary that actually determines which small molecules or other proteins can approach a given region, as opposed to which atoms are merely nearby in 3D. The reduced-surface algorithm of Sanner, Olson, and Spehner (1996) is the standard efficient method for computing this analytically rather than by brute-force sampling, and surface meshes are the natural representation wherever shape complementarity matters directly, most notably molecular docking (Chapter 11).
- **Structural graphs.** A protein represented as a graph — residues (or atoms) as nodes, spatial proximity or covalent bonds as edges, with node/edge features carrying amino acid identity,

dihedral angles, or distances — is the input representation for the graph neural networks used throughout Part III, particularly the equivariant architectures in Chapters 6 and 9 that must respect the same rotation/translation invariance a contact map gets for free. A contact map is, in this light, simply the adjacency matrix of one particular (unweighted, cutoff-thresholded) structural graph — the three representations in this list are not competitors so much as points on a continuum from "discard everything except pairwise topology" (contact maps) to "retain full continuous 3D geometry as graph edge features" (structural graphs).

3.4 Hands-on Project: Structural Feature Extraction & Binding Pocket Geometry

This project computes real spatial features — backbone dihedral angles, a contact map, and binding-pocket residues — directly from a solved PDB structure, using **the same structure Chapter 1 retrieved**: PDB entry `1M17`, the EGFR tyrosine kinase domain in complex with the inhibitor erlotinib (ligand code `AQ4`). Reusing this structure is deliberate — it lets this chapter build directly on the data Chapter 1 already validated, and keeps a single running example (EGFR) coherent across Chapters 1, 3, and the Chapter 16 capstone.

The project computes:

1. **Backbone dihedral angles** (ϕ, ψ) for every residue in chain A, plus a coarse geometry-only secondary-structure call (helix / sheet / coil) from simplified Ramachandran regions — explicitly documented in the code as an approximation, not a DSSP replacement (§3.2). On this structure it gives roughly 40% helix, 38% sheet, 22% coil — a genuinely mixed α/β result, consistent with a kinase domain's fold (a β-sheet-rich N-lobe and α-helix-rich C-lobe), not an artifact of the classifier defaulting to one class.
2. **A Cα–Cα contact map** at an 8 Å cutoff (§3.3), computed as a vectorized pairwise-distance matrix.
3. **Binding pocket residues** — every standard residue with any atom within 5 Å of any atom of the bound ligand (`AQ4` / erlotinib), ranked by distance. On this structure, the closest residue is MET769 at approximately 2.7 Å — a distance in the range of a hydrogen bond, consistent with this inhibitor class's known hinge-region binding mode (this specific distance is a direct geometric computation from the structure, not a claim taken from a secondary source).

See `README.md` for setup and usage, and `structural_features.py` for the implementation. `data/1M17.pdb` is bundled directly in this repository — the same real structure file Chapter 1's project downloads — so the test suite (`tests/test_structural_features.py`) runs fully offline and deterministically; one additional test exercises the live RCSB PDB download path directly, as a reproducibility check on that path specifically.

A note on Google Colab

This project's only non-preinstalled Colab dependency is `biopython` (`numpy` and `requests` both ship on the default runtime); install it with `!pip install biopython` in the first cell. No GPU is required.

References

- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444–2448. https://doi.org/10.1073/pnas.85.8.2444
- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3), 443–453. https://doi.org/10.1016/0022-2836(70)90057-4
- Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195–197. https://doi.org/10.1016/0022-2836(81)90087-5
- Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22), 10915–10919. https://doi.org/10.1073/pnas.89.22.10915
- Sievers, F., Wilm, A., Dineen, D., Gibson, T. J., Karplus, K., Li, W., Lopez, R., McWilliam, H., Remmert, M., Söding, J., Thompson, J. D., & Higgins, D. G. (2011). Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. *Molecular Systems Biology*, 7, 539. https://doi.org/10.1038/msb.2011.75
- Ramachandran, G. N., Ramakrishnan, C., & Sasisekharan, V. (1963). Stereochemistry of polypeptide chain configurations. *Journal of Molecular Biology*, 7, 95–99. https://doi.org/10.1016/S0022-2836(63)80023-6
- Pauling, L., Corey, R. B., & Branson, H. R. (1951). The structure of proteins: two hydrogen-bonded helical configurations of the polypeptide chain. *Proceedings of the National Academy of Sciences*, 37(4), 205–211. https://doi.org/10.1073/pnas.37.4.205
- Kabsch, W., & Sander, C. (1983). Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, 22(12), 2577–2637. https://doi.org/10.1002/bip.360221211
- Senior, A. W., Evans, R., Jumper, J., Kirkpatrick, J., Sifre, L., Green, T., Qin, C., Žídek, A., Nelson, A. W. R., Bridgland, A., Penedones, H., Petersen, S., Simonyan, K., Crossan, S., Kohli, P., Jones, D. T., Silver, D., Kavukcuoglu, K., & Hassabis, D. (2020). Improved protein structure prediction using potentials from deep learning. *Nature*, 577(7792), 706–710. https://doi.org/10.1038/s41586-019-1923-7
- Sanner, M. F., Olson, A. J., & Spehner, J. C. (1996). Reduced surface: an efficient way to compute molecular surfaces. *Biopolymers*, 38(3), 305–320. https://doi.org/10.1002/(SICI)1097-0282(199603)38:3%3C305::AID-BIP4%3E3.0.CO;2-Y

See also Chapter 1′s references for the wwPDB consortium (2019) PDB/ mmCIF citation and Jumper et al. (2021) AlphaFold2 citation, both reused here rather than re-listed.

All citations above were verified against PubMed records before use (see this repository's editorial process); the structural data cited in §3.4 (residue count, secondary-structure fractions, pocket residues and distances) was computed directly from the bundled `data/1M17.pdb` file, not taken from a secondary source — see `structural_features.py` to reproduce.